Конашинская Ольга Александровна 090303-Пиб-о24

Лабораторная работа №6

Вариант: № 10

Тема: Использование декораторов, контекстных менеджеров, dataclass, аннотаций типов, перечислений и механизмов интроспекции в Python.

Постановка задачи:

Необходимо разработать консольное приложение для управления объектами выбранной предметной области согласно индивидуальному варианту.

Приложение должно содержать:

1. несколько типов объектов;
2. перечисление состояний объектов с использованием Enum;
3. классы данных с использованием dataclass;
4. аннотации типов;
5. собственные декораторы;
6. хотя бы один декоратор с параметрами;
7. собственный контекстный менеджер;
8. Protocol;
9. механизм интроспекции объектов;
10. консольное меню;
11. обработку исключений;
12. формирование итоговой статистики.

Программа должна предоставлять пользователю меню. Не обязательное в консоли, можно использовать графические меню.

Минимальный набор операций:

1. Добавить объект
2. Удалить объект
3. Найти объект
4. Вывести все объекты
5. Изменить состояние объекта
6. Выполнить операцию предметной области
7. Получить статистику
8. Выполнить интроспекцию объекта
9. Демонстрация декораторов
10. Демонстрация контекстного менеджера
0. Выход

Краткие теоретические сведения:

Типизация и структурирование данных

- Аннотации типов: Позволяют указывать ожидаемые типы аргументов и возвращаемых значений функций (price: float). Программа не проверяет их при запуске, но они нужны для читаемости, IDE и статических анализаторов.
- dataclass: Декоратор @dataclass автоматически генерирует конструкторы (__init__), методы сравнения и строковое представление для классов, которые созданы в основном для хранения данных.
- Enum: Классы перечислений применяются для хранения фиксированного набора неизменяемых значений (например, статусов заказа), заменяя собой обычные строки.
- Protocol: Позволяет реализовать «утиную типизацию» на уровне статического анализа. Объект считается соответствующим протоколу, если в нем реализованы описанные методы, при этом наследоваться от базового класса не нужно.

Управление поведением (Декораторы)

- Декораторы: Функции, которые оборачивают другие функции, позволяя изменять или расширять их поведение без модификации основного кода.
- functools.wraps: Декоратор, который обязательно нужно использовать внутри своего декоратора, чтобы сохранить метаданные (имя, документацию) исходной функции.
- Декораторы с параметрами: Создаются через три уровня вложенности функций, где внешний уровень принимает настройки для самого декоратора (например, количество повторов).

Управление ресурсами и интроспекция

- Контекстные менеджеры (with): Гарантируют выполнение кода при входе и выходе из блока (например, автоматическое закрытие файлов). Создаются через методы класса __enter__ / __exit__.
- Декоратор @contextmanager: Позволяет быстро создать контекстный менеджер из обычной функции-генератора с использованием try ... finally и конструкции yield.

- Интроспекция: Инструменты для анализа структуры объектов во время работы программы (проверка типов через isinstance, получение атрибутов через getattr/hasattr). Модуль inspect расширяет это, позволяя читать сигнатуры функций.

Исходный код программы:

main.py

```
import models
import services
from models import Train, Carriage, Ticket, TicketStatus
from context import Operation
from decorators import log_call, repeat

trains: list[Train] = [
    Train(1, "001A", "Москва - Санкт-Петербург",
          [Carriage(1, 1, 10), Carriage(2, 2, 10)]),
    Train(2, "055B", "Москва - Казань",
          [Carriage(3, 1, 8)]),
]

tickets: list[Ticket] = [
    Ticket(1, "Иванов И.И.", "001A", 1, 5, 2500, TicketStatus.PAID),
    Ticket(2, "Петров П.П.", "001A", 1, 6, 2500,
TicketStatus.RESERVED),
    Ticket(3, "Сидоров С.С.", "055B", 3, 2, 1800, TicketStatus.USED),
]

next_id = 4


def demo() -> None:
    print("\n--- Демонстрация возможностей ---")
    print("dataclass:", tickets[0])
    print("Enum:", TicketStatus.PAID, "=", TicketStatus.PAID.value)
    train = trains[0]
    print("Protocol (get_report_data):")
    services.print_report(train)
    services.print_report(tickets[0])
    services.inspect_object(tickets[0])
    print("--- Конец демонстрации ---\n")


def main() -> None:
    global next_id
    demo()
    while True:
        print("===== МЕНЮ =====",
```

```
              "1 - Добавить билет",
              "2 - Удалить билет",
              "3 - Найти билет",
              "4 - Вывести все билеты",
              "5 - Изменить статус билета",
              "6 - Расчёт загрузки поезда / свободные места",
              "7 - Статистика",
              "8 - Интроспекция билета",
              "9 - Демонстрация декораторов",
              "10 - Демонстрация контекстного менеджера",
              "0 - Выход", sep="\n")
        choice = input("Выберите пункт меню: ")

        if choice == "1":
            try:
                passenger = input("ФИО пассажира: ")
                train_number = input("Номер поезда: ")
                carriage_number = int(input("Номер вагона: "))
                seat = int(input("Место: "))
                price = float(input("Цена: "))
                ticket = Ticket(next_id, passenger, train_number,
carriage_number, seat, price)
                with Operation("добавление билета"):
                    services.add_ticket(tickets, ticket)
                next_id += 1
                print("Билет добавлен.")
            except models.SeatTakenError as e:
                print("Ошибка:", e)
            except ValueError:
                print("Ошибка: введено не число.")

        elif choice == "2":
            try:
                ticket_id = int(input("ID билета: "))
                services.remove_ticket(tickets, ticket_id)
                print("Билет удалён.")
            except models.TicketNotFoundError as e:
                print("Ошибка:", e)
            except ValueError:
                print("Ошибка: введено не число.")

        elif choice == "3":
            try:
                ticket_id = int(input("ID билета: "))
                print(services.find_ticket(tickets,
ticket_id).get_report_data())
            except models.TicketNotFoundError as e:
                print("Ошибка:", e)
            except ValueError:
                print("Ошибка: введено не число.")
```

```
        elif choice == "4":
            for t in tickets:
                print(t.get_report_data())

        elif choice == "5":
            try:
                ticket_id = int(input("ID билета: "))
                ticket = services.find_ticket(tickets, ticket_id)
                print("Статусы:", ", ".join(s.name for s in
TicketStatus))
                name = input("Новый статус: ").upper()
                services.change_status(ticket, TicketStatus[name])
                print("Статус изменён.")
            except models.TicketNotFoundError as e:
                print("Ошибка:", e)
            except KeyError:
                print("Ошибка: нет такого статуса.")
            except ValueError as e:
                print("Ошибка:", e)

        elif choice == "6":
            number = input("Номер поезда: ")
            train = services.find_train(trains, number)
            if train is None:
                print("Поезд не найден.")
            else:
                try:
                    load = services.train_load(train, tickets)
                    print(f"Загрузка поезда: {load:.1f}%")
                    print("Свободные места по вагонам:")
                    for carriage, seats in services.free_seats(train,
tickets).items():
                        print(f"  Вагон {carriage}: {seats}")
                except ZeroDivisionError as e:
                    print("Ошибка:", e)

        elif choice == "7":
            for key, value in
services.get_statistics(tickets).items():
                print(f"{key}: {value}")

        elif choice == "8":
            try:
                ticket_id = int(input("ID билета: "))
                services.inspect_object(services.find_ticket(tickets,
ticket_id))
            except models.TicketNotFoundError as e:
                print("Ошибка:", e)
            except ValueError:
```

```
                print("Ошибка: введено не число.")

        elif choice == "9":
            @log_call
            def sample_sum(a: int, b: int) -> int:
                return a + b

            @repeat(3)
            def hello() -> None:
                print("Привет из декоратора repeat(3)")

            sample_sum(2, 3)
            hello()

        elif choice == "10":
            with Operation("демонстрация контекстного менеджера"):
                print("Выполняется работа внутри блока with...")

        elif choice == "0":
            print("Программа завершена!")
            break
        else:
            print("Некорректный ввод.")


if __name__ == "__main__":
    main()
```

operations.py

```
from models import *

def new_train(id_, route):
    train = []
    railcars = []
    seats = []

    train_ = Train(id_, route)
    for coupe in range(train_.coupe_num):
        railcars.append(Railcar(coupe + 1, RailType.COUPE, 36, id_))
    for platz in range(train_.platz_num):
        railcars.append(Railcar(platz + 1, RailType.PLATZCART, 54,
id_))

    for rail in range(1, len(railcars) + 1):
        for seat_ in range(1, railcars[rail - 1].capacity + 1):
            if railcars[rail - 1].rail_type == RailType.COUPE:
                if seat_ % 2 == 0:
                    seats.append(Seat(seat_, SeatType.UP,
railcars[rail - 1].id))
                else:
```

```
                    seats.append(Seat(seat_, SeatType.DOWN,
railcars[rail - 1].id))
            else:
                if seat_ % 4 == 1:
                    seats.append(Seat(seat_, SeatType.UP,
railcars[rail - 1].id))
                elif seat_ % 4 == 2:
                    seats.append(Seat(seat_, SeatType.DOWN,
railcars[rail - 1].id))
                elif seat_ % 4 == 3:
                    seats.append(Seat(seat_, SeatType.SIDE_UP,
railcars[rail - 1].id))
                elif seat_ % 4 == 0:
                    seats.append(Seat(seat_, SeatType.SIDE_DOWN,
railcars[rail - 1].id))

    train.append(train_)
    train.append(railcars)
    train.append(seats)
    return train
```

models.py

```
from dataclasses import dataclass, field
from enum import Enum

class TicketStatus(Enum):
    RESERVED = "забронирован"
    PAID = "оплачен"
    USED = "использован"
    RETURNED = "возвращён"


class TicketNotFoundError(Exception):
    pass

class SeatTakenError(Exception):
    pass

@dataclass
class Entity:
    id: int

@dataclass
class Carriage(Entity):
    number: int
    seats: int


@dataclass
class Train(Entity):
```

```
    number: str
    route: str
    carriages: list[Carriage] = field(default_factory=list)

    def total_seats(self) -> int:
        return sum(c.seats for c in self.carriages)

    def get_report_data(self) -> str:
        return f"Поезд №{self.number} ({self.route}), вагонов:
{len(self.carriages)}, мест: {self.total_seats()}"

@dataclass
class Ticket(Entity):
    passenger: str
    train_number: str
    carriage_number: int
    seat: int
    price: float
    status: TicketStatus = TicketStatus.RESERVED

    def get_report_data(self) -> str:
        return f"Билет #{self.id}: {self.passenger}, поезд
{self.train_number}, вагон {self.carriage_number}, место {self.seat} —
{self.status.value}"
```

context.py

```
import time

class Operation:
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        print(f"[OPERATION] Начало операции: {self.name}")
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        elapsed = time.perf_counter() - self.start
        print(f"[OPERATION] Завершение операции: {self.name}
({elapsed:.6f} sec)")
        if exc_type is not None:
            print(f"[OPERATION] Произошла ошибка: {exc_value}")
        return False
```

decorators.py

```
import time
from functools import wraps
```

```
def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[LOG] вызван метод {func.__name__}()")
        result = func(*args, **kwargs)
        print(f"[LOG] {func.__name__}() вернул: {result}")
        return result
    return wrapper

def measure_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"[PERFORMANCE] {func.__name__}: {elapsed:.6f} sec")
        return result
    return wrapper

def repeat(count):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(count):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator
```

protocols.py

```
from typing import Protocol

class Reportable(Protocol):
    def get_report_data(self) -> str:
        ...
```

services.py

```
import inspect
from models import Ticket, Train, TicketStatus, TicketNotFoundError,
SeatTakenError
from protocols import Reportable
from decorators import log_call, measure_time, repeat


@log_call
def add_ticket(tickets: list[Ticket], ticket: Ticket) -> int:
    for t in tickets:
        same_seat = (t.train_number == ticket.train_number and
```

```
                    t.carriage_number == ticket.carriage_number and
                    t.seat == ticket.seat)
        if same_seat and t.status != TicketStatus.RETURNED:
            raise SeatTakenError(f"Место {ticket.seat} в вагоне
{ticket.carriage_number} уже занято")
    tickets.append(ticket)
    return ticket.id


@log_call
def remove_ticket(tickets: list[Ticket], ticket_id: int) -> bool:
    ticket = find_ticket(tickets, ticket_id)
    tickets.remove(ticket)
    return True


def find_ticket(tickets: list[Ticket], ticket_id: int) -> Ticket:
    for t in tickets:
        if t.id == ticket_id:
            return t
    raise TicketNotFoundError(f"Билет #{ticket_id} не найден")


def find_train(trains: list[Train], number: str) -> Train | None:
    for tr in trains:
        if tr.number == number:
            return tr
    return None


def change_status(ticket: Ticket, new_status: TicketStatus) -> None:
    if ticket.status == TicketStatus.RETURNED:
        raise ValueError("Возвращённый билет нельзя изменить")
    ticket.status = new_status


@measure_time
def train_load(train: Train, tickets: list[Ticket]) -> float:
    busy = [t for t in tickets
            if t.train_number == train.number and t.status !=
TicketStatus.RETURNED]
    total = train.total_seats()
    if total == 0:
        raise ZeroDivisionError("У поезда нет вагонов с местами")
    return len(busy) / total * 100


def free_seats(train: Train, tickets: list[Ticket]) -> dict[int,
list[int]]:
    result: dict[int, list[int]] = {}
```

```
    for c in train.carriages:
        taken = {t.seat for t in tickets
                 if t.train_number == train.number
                 and t.carriage_number == c.number
                 and t.status != TicketStatus.RETURNED}
        result[c.number] = [s for s in range(1, c.seats + 1) if s not
in taken]
    return result


def get_statistics(tickets: list[Ticket]) -> dict:
    total = len(tickets)
    paid = len([t for t in tickets if t.status == TicketStatus.PAID])
    used = len([t for t in tickets if t.status == TicketStatus.USED])
    reserved = len([t for t in tickets if t.status ==
TicketStatus.RESERVED])
    returned = len([t for t in tickets if t.status ==
TicketStatus.RETURNED])
    prices = [t.price for t in tickets if t.status !=
TicketStatus.RETURNED]
    revenue = sum(prices)
    average = revenue / len(prices) if prices else 0
    return {
        "Всего билетов": total,
        "Забронировано": reserved,
        "Оплачено": paid,
        "Использовано": used,
        "Возвращено": returned,
        "Выручка": revenue,
        "Средняя цена": round(average, 2),
    }


@repeat(2)
def print_report(obj: Reportable) -> None:
    print("  >>", obj.get_report_data())


def inspect_object(obj) -> None:
    print("=== ИНФОРМАЦИЯ ОБ ОБЪЕКТЕ ===")
    print("Тип:", type(obj).__name__)
    print("Это билет?", isinstance(obj, Ticket))
    print("Атрибуты:")
    for name in dir(obj):
        if name.startswith("_"):
            continue
        value = getattr(obj, name)
        if not callable(value):
            print(f"    {name} = {value}")
    print("Методы:")
```

```
    for name in dir(obj):
        if name.startswith("_"):
            continue
        member = getattr(obj, name)
        if callable(member):
            print(f"    {name}{inspect.signature(member)}")
```

```
--- Демонстрация возможностей ---
dataclass: Ticket(id=1, passenger='Иванов И.И.', train_number='001A', carriage_number=1, seat=5, price=2500, status=<TicketStatus.PAID: 'оплачен'>)
Enum: TicketStatus.PAID = оплачен
Protocol (get_report_data):
  >> Поезд №001A (Москва - Санкт-Петербург), вагонов: 2, мест: 20
  >> Поезд №001A (Москва - Санкт-Петербург), вагонов: 2, мест: 20
  >> Билет #1: Иванов И.И., поезд 001A, вагон 1, место 5 — оплачен
  >> Билет #1: Иванов И.И., поезд 001A, вагон 1, место 5 — оплачен
=== ИНФОРМАЦИЯ ОБ ОБЪЕКТЕ ===
Тип: Ticket
Это билет? True
Атрибуты:
    carriage_number = 1
    id = 1
    passenger = Иванов И.И.
    price = 2500
    seat = 5
    status = TicketStatus.PAID
    train_number = 001A
Методы:
    get_report_data() -> str
--- Конец демонстрации ---
```

```
===== МЕНЮ =====
1 - Добавить билет
2 - Удалить билет
3 - Найти билет
4 - Вывести все билеты
5 - Изменить статус билета
6 - Расчёт загрузки поезда / свободные места
7 - Статистика
8 - Интроспекция билета
9 - Демонстрация декораторов
10 - Демонстрация контекстного менеджера
0 - Выход
Выберите пункт меню: 1
ФИО пассажира: Лупин Заур Александрович
Номер поезда: 001A
Номер вагона: 1
Место: 1
Цена: 1500
[OPERATION] Начало операции: добавление билета
[LOG] вызван метод add_ticket()
[LOG] add_ticket() вернул: 4
[OPERATION] Завершение операции: добавление билета (0.000040 sec)
Билет добавлен.
```

```
Выберите пункт меню: 2
ID билета: 1
[LOG] вызван метод remove_ticket()
[LOG] remove_ticket() вернул: True
Билет удалён.
```

```
Выберите пункт меню: 3
ID билета: 2
Билет #2: Петров П.П., поезд 001A, вагон 1, место 6 — забронирован
```

```
Выберите пункт меню: 4
Билет #2: Петров П.П., поезд 001A, вагон 1, место 6 — забронирован
Билет #3: Сидоров С.С., поезд 055B, вагон 3, место 2 — использован
Билет #4: Лупин Заур Александрович, поезд 001A, вагон 1, место 1 — забронирован
```

```
Выберите пункт меню: 5
ID билета: 2
Статусы: RESERVED, PAID, USED, RETURNED
Новый статус: USED
Статус изменён.
```

```
Выберите пункт меню: 6
Номер поезда: 001A
[PERFORMANCE] train_load: 0.000029 sec
Загрузка поезда: 10.0%
Свободные места по вагонам:
  Вагон 1: [2, 3, 4, 5, 7, 8, 9, 10]
  Вагон 2: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
Выберите пункт меню: 7
Всего билетов: 3
Забронировано: 1
Оплачено: 0
Использовано: 2
Возвращено: 0
Выручка: 5800.0
Средняя цена: 1933.33
```

```
Выберите пункт меню: 8
ID билета: 2
=== ИНФОРМАЦИЯ ОБ ОБЪЕКТЕ ===
Тип: Ticket
Это билет? True
Атрибуты:
    carriage_number = 1
    id = 2
    passenger = Петров П.П.
    price = 2500
    seat = 6
    status = TicketStatus.USED
    train_number = 001A
Методы:
    get_report_data() -> str
```

```
Выберите пункт меню: 9
[LOG] вызван метод sample_sum()
[LOG] sample_sum() вернул: 5
Привет из декоратора repeat(3)
Привет из декоратора repeat(3)
Привет из декоратора repeat(3)
```

```
Выберите пункт меню: 10
[OPERATION] Начало операции: демонстрация контекстного менеджера
Выполняется работа внутри блока with...
[OPERATION] Завершение операции: демонстрация контекстного менеджера (0.000012 sec)
```